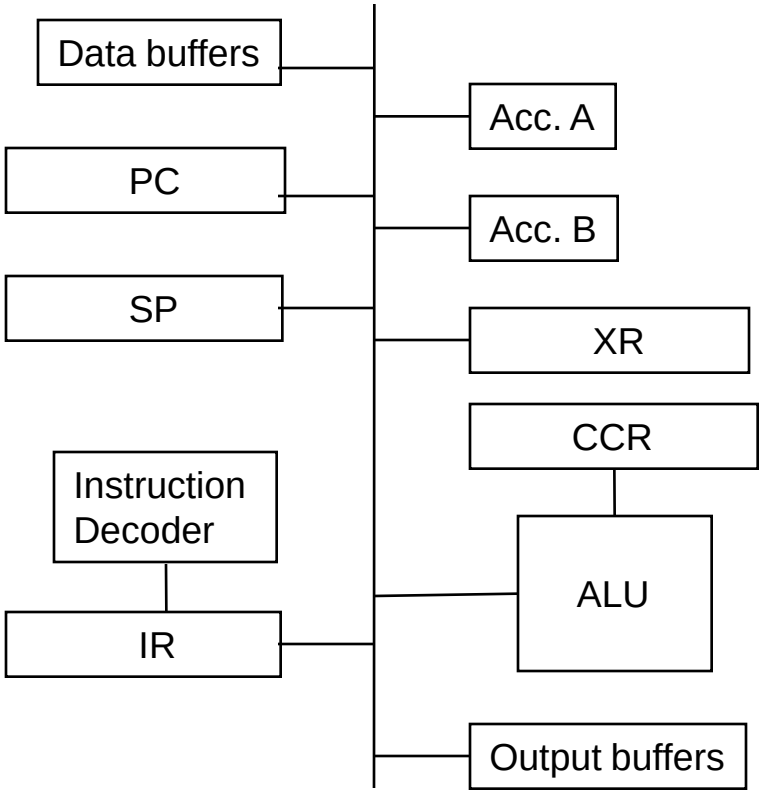
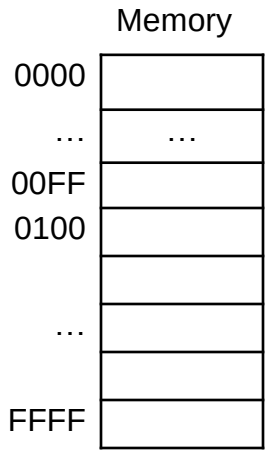
Assembly Language Programming

M6800


Data buffers
Acc. A
PC
Acc. B
SP
XR
CCR
Instruction Decoder
ALU
IR
Output buffers

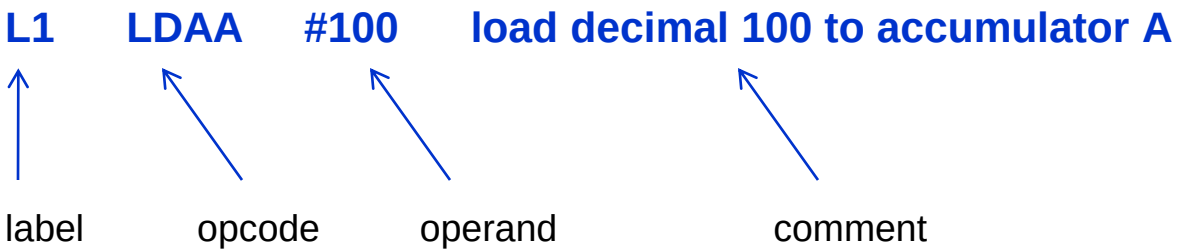
M6800

Acc. A	8 bits
Acc. B	8 bits
PC	16 bits
XR	16 bits
SP	16 bits
Data buffers	8 bits



Addressing Modes

Instruction Format



Addressing Modes

Immediate Addressing

3+5

```
LDAA    #3
ADDA    #5
```

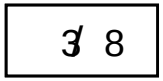
A | 3 / 8

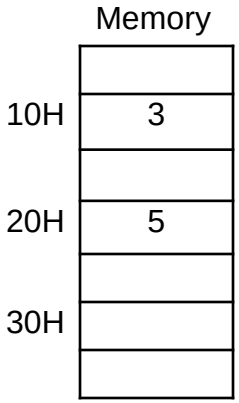
, D	decimal	3	3**D**
%, B	binary	%00110101	00110101**B**
$, H	hexadecimal	$2F	2F**H**
@, C	octal	@123	123**C**

Addressing Modes

Direct Addressing

```
LDAA    10H
ADDA    20H
STAA    30H
```

A 



Ex Increment the value at location 50_H by 1.

```
LDAA    50H
ADDA    #1
STAA    50H
```

```
LDAA    50H
INCA
STAA    50H
```

Implied addressing

```
INC     50H
```

Addressing Modes

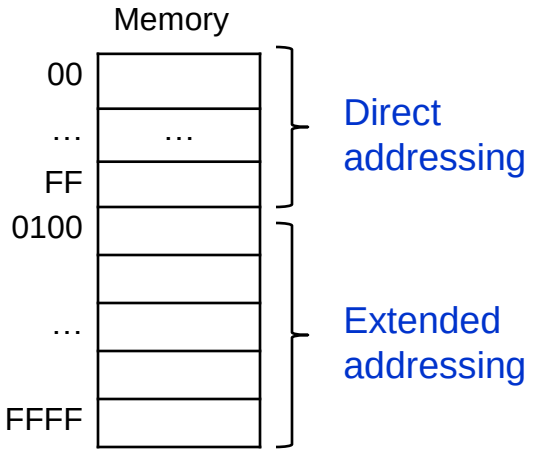
Ex $M[4C_H] \leftarrow \overline{3F}_H \oplus M[20_H]$

```
LDAB   #3FH          C6 3F
COMB                 53
EORB   20H           D8 20
STAB   4CH           D7 4C
```

Extended Addressing

Ex $M[\mathbf{2000}_H] \leftarrow \overline{3F}_H \oplus M[20_H]$

```
LDAB   #3FH          C6 3F
COMB                 53
EORB   20H           D8 20
STAB   2000H         F7 2000
```



Instructions

Add
ADDA
ADDB
ABA
ADCA
ADCB

Subtract
SUBA
SUBB
SBA
SBCA
SBCB

Load
LDAA
LDAB

Store
STAA
STAB

AND
ANDA
ANDB

OR
ORAA
ORAB

XOR
EORA
EORB

Transfer Accumulator
TAB
TBA

Clear
CLRA
CLRB
CLR

Increment
INCA
INCB
INC

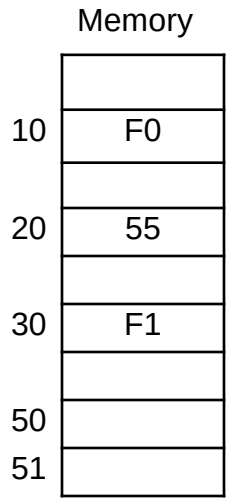
Decrement
DECA
DECB
DEC

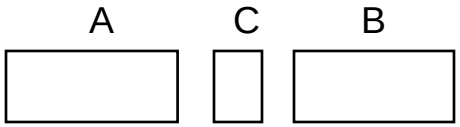
1's complement
COMA
COMB
COM

Negate (2's complement)
NEGA
NEGB
NEG

Instructions

Ex Add with carry





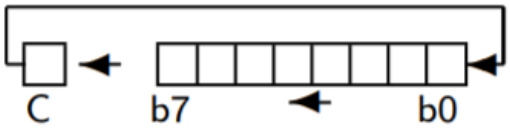
		A	C	B
CLRA		00		
LDAB	10H			F0
ADDB	20H		1	45
ADCA	#0	01		45
ADDB	30H		1	36
ADCA	#0	02		36
STAA	50H			
STAB	51H			

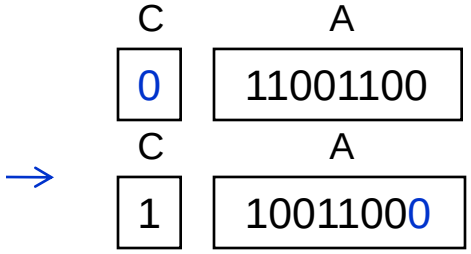
ADCA A←A+C+M

Instructions

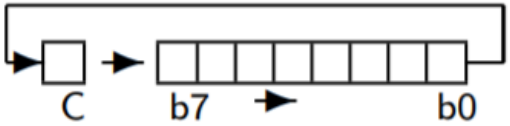
Shift/Rotate

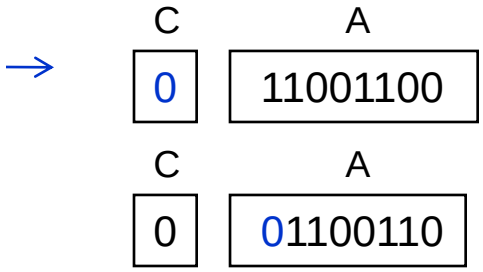
Rotate left





Rotate right





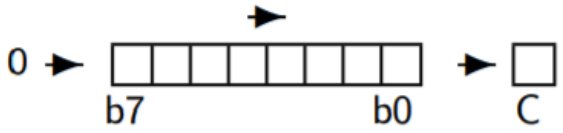
ROLA
ROLB
ROL

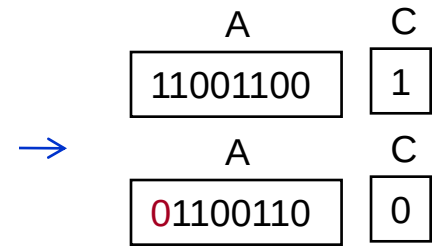
RORA
RORB
ROR

Instructions

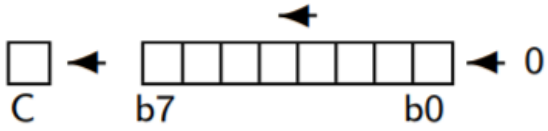
Shift/Rotate

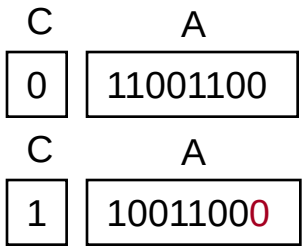
Logical shift right
LSRA
LSRB
LSR



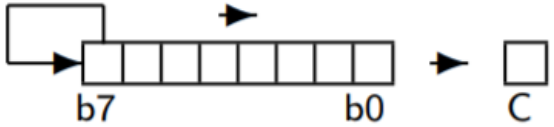


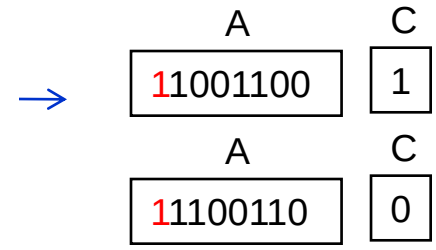
Arithmetic shift left
ASLA
ASLB
ASL





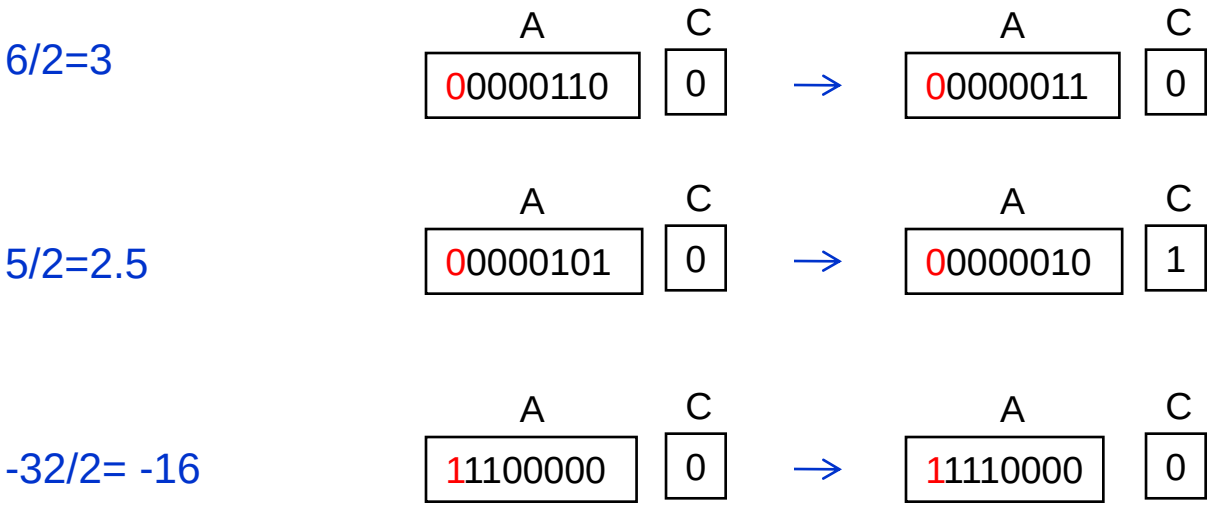
Arithmetic shift right
ASRA
ASRB
ASR





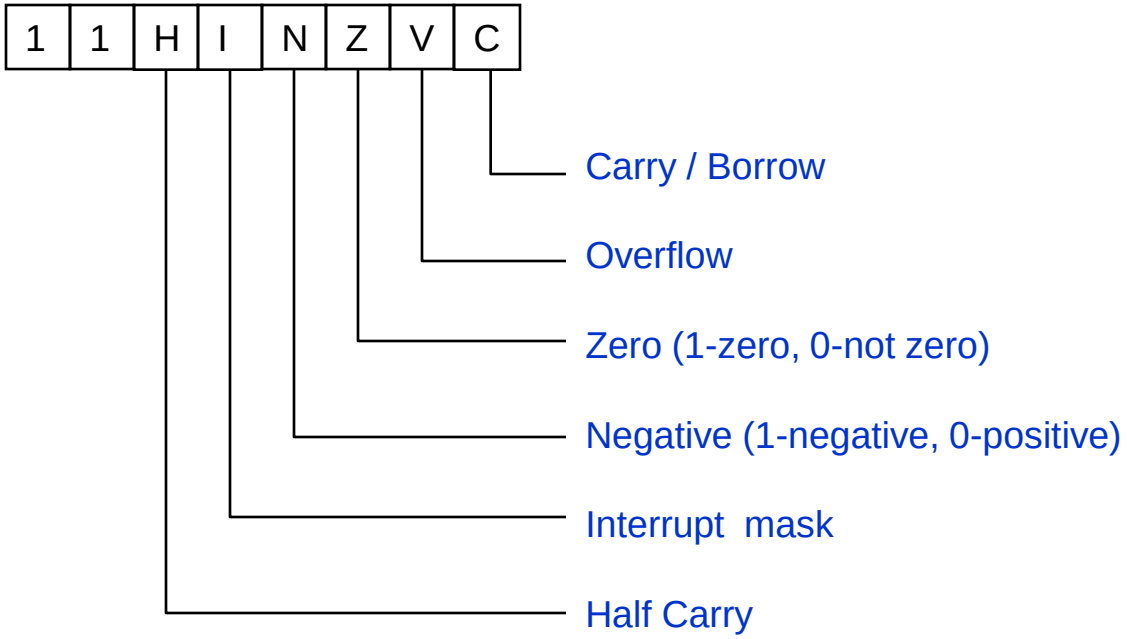
Instructions

Arithmetic Shift Right



Instructions

Condition Code Register (CCR)



Instructions

Overflow

CCR	1	1	H	I	N	Z	**V**	C

Decimal	Binary
-128	10000000
-127	10000001
-126	10000010
…	…
-4	11111100
-3	11111101
-2	11111110
-1	11111111
0	00000000
1	00000001
2	00000010
3	00000011
…	…
125	01111101
126	01111110
127	01111111

Instructions

Ex: $79_{10} + 64_{10} = 143_{10}$

```
79        0100 1111
64      + 0100 0000
        ------------
          0 1000 1111
          C    A
```

N=1

C=0

V=1

H=0

Z=0

			H		N	Z	V	C
CCR	1	1	0	I	1	0	**1**	0

Instructions

Ex: $-79_{10} + -64_{10} = -143_{10}$

-79 1011 0001

-64 + 1100 0000

1 0111 0001

C A

N=0

C=1

V=1

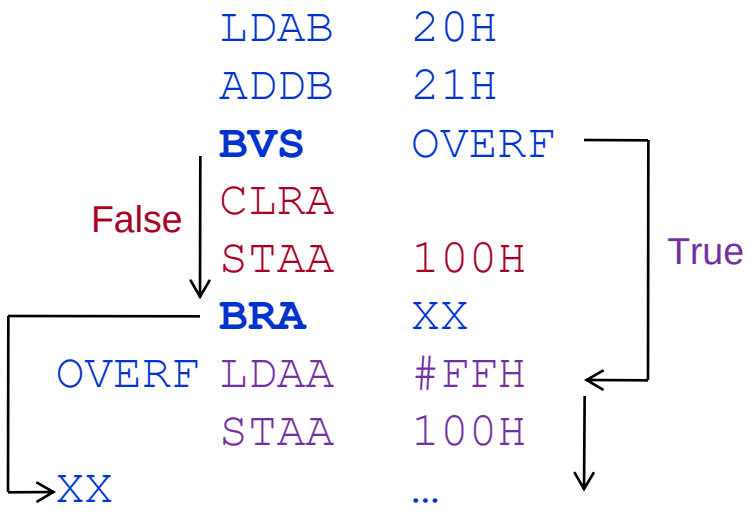
H=0

Z=0

	H		N	Z	V	C
CCR	1 1	0	I	0	0	**1**	1

Instructions

Ex: Add the numbers at locations 20_H and 21_H. If there is an overflow store FF_H at location 100_H, otherwise store 0 at the same location.



Instructions

Ex: Subtract 5 from the number at location 100_H.
If the result is equal to 0, store FF_H at location 200_H ;
If the result is greater than 0, store 1 at location 200_H ;
If the result is less than 0, store 0 at location 200_H.

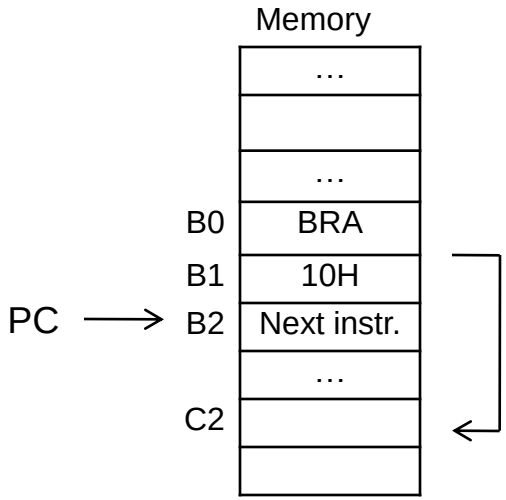
```
        LDAB    100H
        SUBB    #5H
        BEQ     ZERO
        BPL     POS
        BMI     NEG
        BRA     XX
ZERO    LDAA    #FFH
        BRA     STORE
POS     LDAA    #1
        BRA     STORE
NEG     CLRA
STORE   STAA    200H
XX      …
```

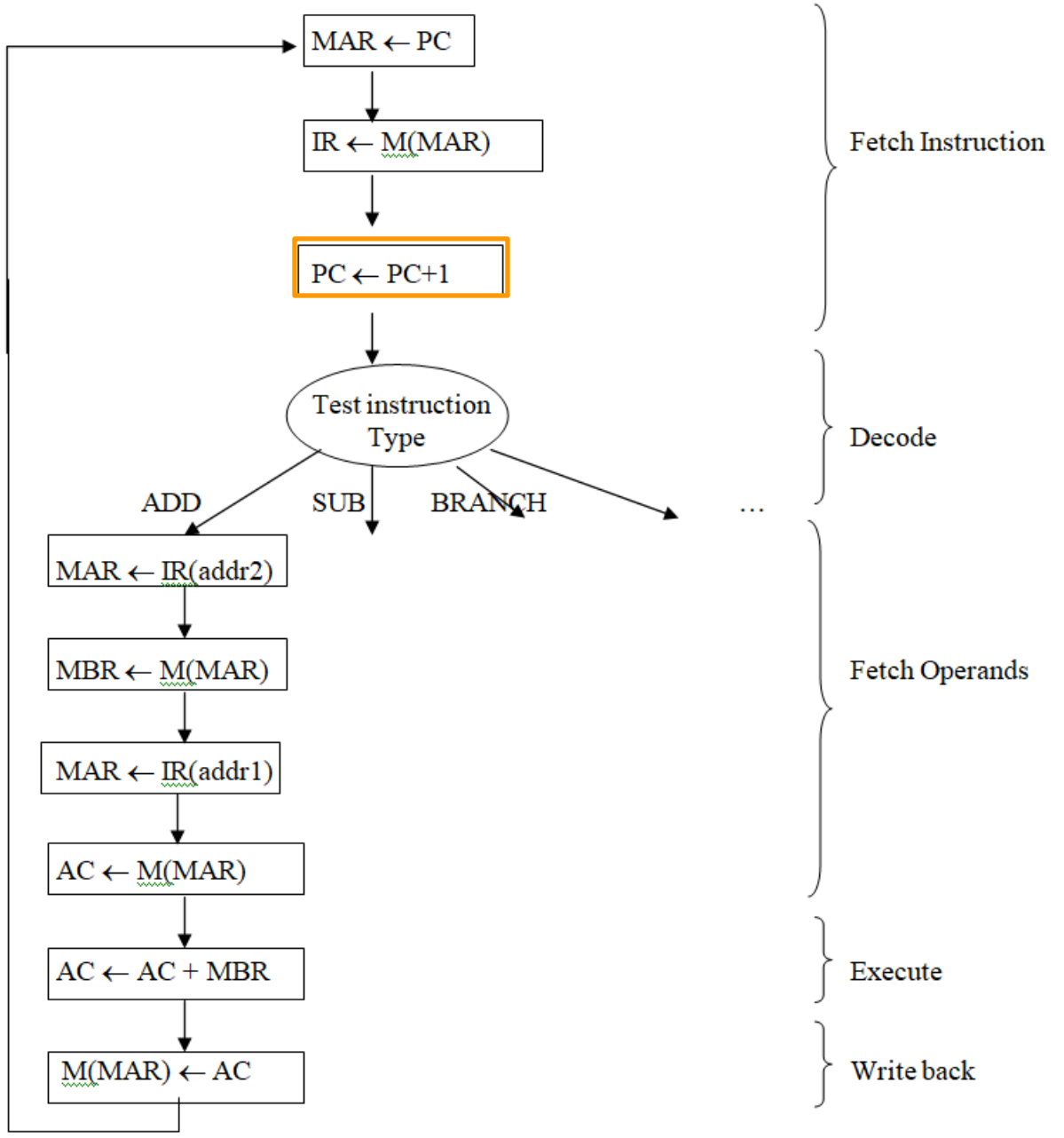
Instructions

Relative Addressing

EA = PC + offset

$B2_H + 10_H = C2_H$





Instruction Execution Cycle

1. Fetch instruction
2. Decode instruction
3. Fetch operands
4. Execute instruction
5. Write back

Instructions

Relative Addressing

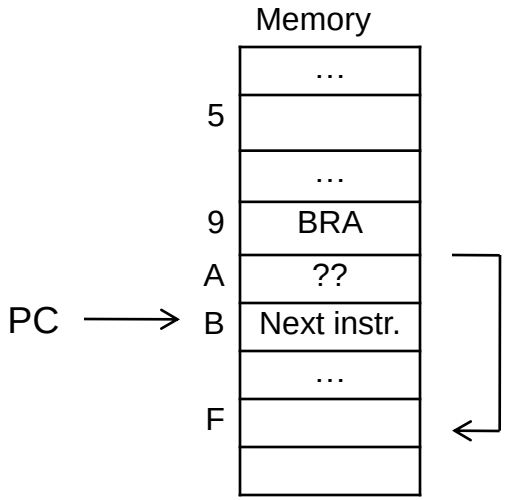
Forward reference

EA = PC + offset

$F_H = B_H + x$

$x = F_H - B_H$

$x = 4_H$



Instructions

Relative Addressing

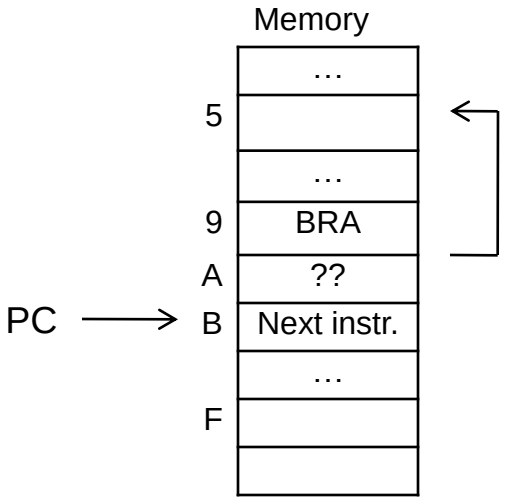
Backward reference

EA = PC + offset

$5_H = B_H + x$

$x = 5_H - B_H = 5_H + (-B_H)$

$x = FA_H$



5 0000 0000 0000 0101

-B + 1111 1111 1111 0101

1111 1111 1111 1010

FA

Instructions

Compare

CMPA

CMPB

Ex: Branch if accumulator A is equal to 5

```
SUBA    #5
BEQ     XX
```

```
SUBA:   A←A-5
```

```
CMPA    #5
BEQ     XX
```

```
CMPA:   A-5
```

Instructions

Bit test

BITA
BITB

ANDA: $A \leftarrow A \wedge 01_H$

BITA: $A \wedge 01_H$

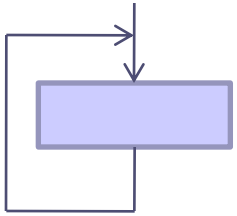
Ex: If accumulator A is even, store 0 at location 100_H, otherwise store FF_H at the same location.

even	odd
xxxxxxx0	xxxxxxx1
∧ 00000001	∧ 00000001
00000000	00000001

```
        BITA  #01
        BEQ   EVEN
        LDAB  #FFH
        BRA   STORE
EVEN    CLRB
STORE   STAB  100H
```

Loops

Infinite Loop

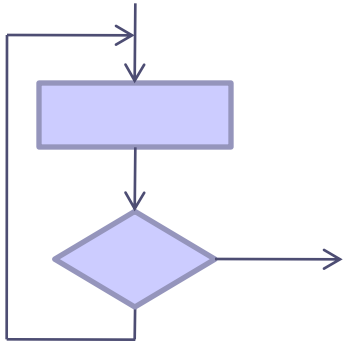


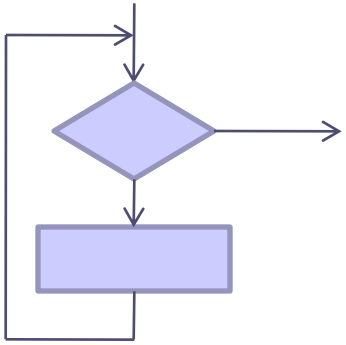
Ex: Implement a counter that counts as 0,1,2, … FF, 0,1 2 …

```
        CLRA
LOOP    INCA
        BRA     LOOP
```

Loops

Conditional Loop





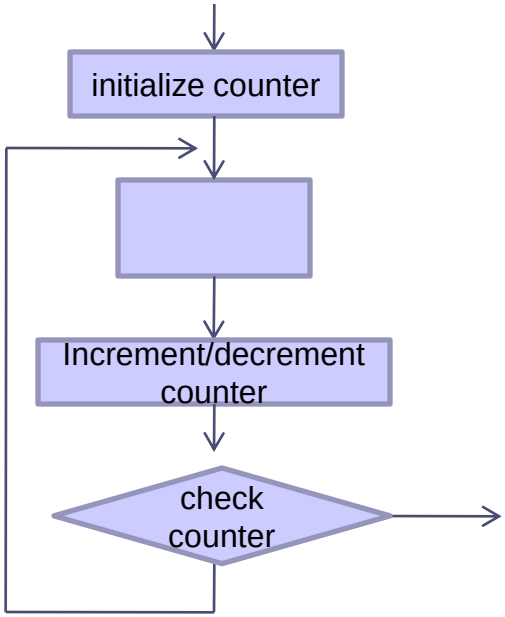
Ex: Load the contents of location 100_H into accumulator A. Decrement it by 1 until it is equal to 0.

```
     LDAA    100H
L1   DECA
     BNE     L1
```

```
     LDAA    100H
L1   BEQ     XX
     DECA
     BRA     L1
XX   …
```

Loops

Counter Controlled Loop



Ex: Increment the value at location 100_H 32 times.

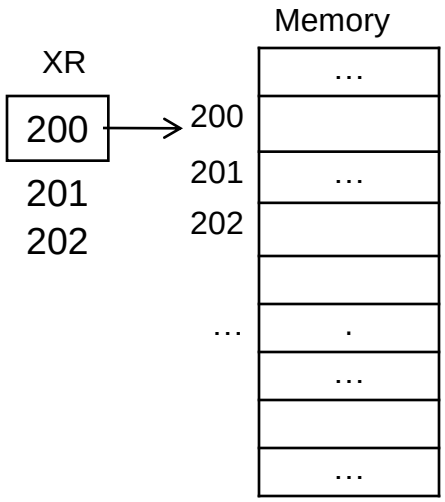
```
        LDAA    #0
L1      INC     100H
        INCA
        CMPA    #32
        BEQ     XX
        BRA     L1
XX      …
```

or

```
        LDAA    #32
L1      INC     100H
        DECA
        BNE     L1
```

Addressing Modes

Indexed Addressing

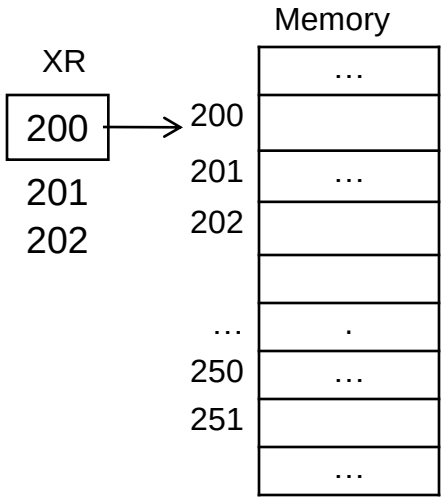


Ex1: Array A starts from 200_H and has 10 elements. Add 5 to each element of A.

```
      LDX     #200H
      LDAB    #10
L1    LDAA    0,X       EA = XR + offset
      ADDA    #5
      STAA    0,X
      INX
      DECB
      BNE     L1
```

Addressing Modes

Indexed Addressing

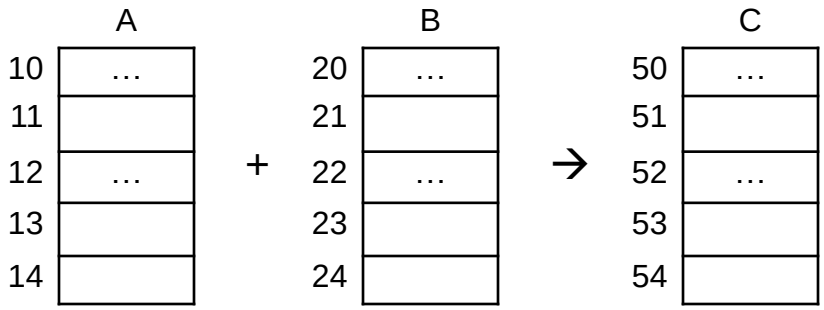


Ex2: Array A starts from 200_H and has 10 elements. Copy it to another location starting from 250_H.

```
        LDX     #200H
        LDAB    #10
L1      LDAA    0,X
        STAA    50H,X    EA = XR + offset
        INX
        DECB
        BNE     L1
```

Addressing Modes

Indexed Addressing

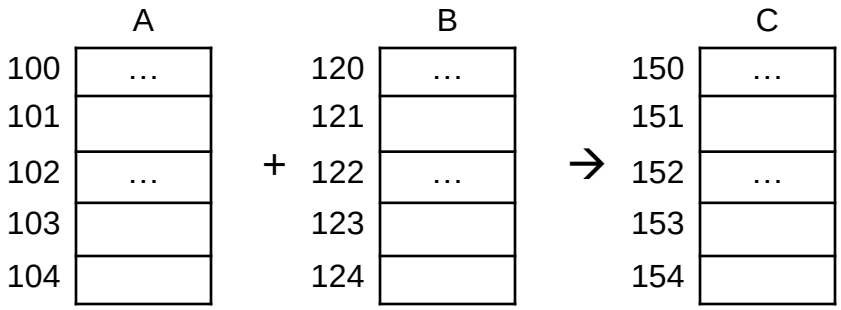


Ex3: A+B→C

```
        LDX     #0H
        LDAB    #5
L1      LDAA    10H,X
        ADDA    20H,X
        STAA    50H,X
        INX
        DECB
        BNE     L1
```

Addressing Modes

Indexed Addressing

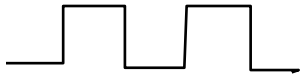


Ex3: A+B→C

```
        LDX     #100H
        LDAB    #5
L1      LDAA    0H,X
        ADDA    20H,X
        STAA    50H,X
        INX
        DECB
        BNE     L1
```

Time Delays

Clock Speed



- clock frequency (f)

  1MHz : 1 000 000 cycles/second

- cycle time (t)

  t = 1/f

  1/1MHz = 10^{-6} s = 1µs

Hz		1
KHz	Kilo	10^{3}
MHz	Mega	10^{6}
GHz	Giga	10^{9}

second		1
ms	milli	10^{-3}
µs	micro	10^{-6}
ns	nano	10^{-9}

Time Delays

Ex: 8-bit register

```
                          cycles
      LDAA  #10             2
LOOP  DECA                  2
      BNE   LOOP            4
```

T = 2+10*(2+4) = 62 cycles = 62μs

Ex:

```
                          cycles
      LDAA  #FFH            2
LOOP  DECA                  2
      BNE   LOOP            4
```

Ex:

```
                          cycles
      LDAA  #0              2
LOOP  DECA                  2
      BNE   LOOP            4
```

T = 2+2^8*(2+4) = 1538 cycles ≈ 1.5ms

Time Delays

Ex: Two 8-bit registers

```
                          cycles
     LDAA   #0              2
L1   LDAB   #0              2
L2   DECB                   2
     BNE    L2              4
     DECA                   2
     BNE    L1              4
```

$T = 2 + 2^8 * (2 + 2^8 * (2+4) + 2 + 4) \approx 0.4$ s

Time Delays

Ex: 16-bit register

```
                          cycles
     LDX    #0000          3
L1   DEX                   4
     BNE    L1             4
```

$T = 3 + 2^{16} * (4+4) \approx 0.52$ s

Time Delays

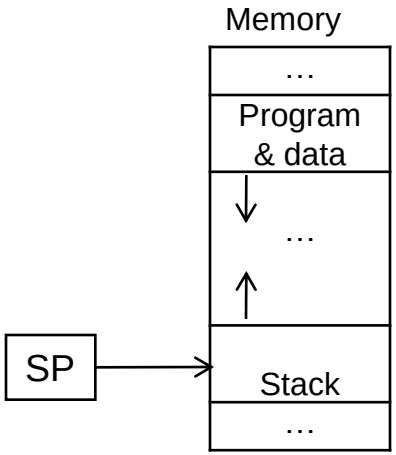
Ex: Generate 300ms delay

```
                          cycles
     LDX     #??H          3
L1   DEX        ┐          4
     BNE     L1 ┘          4
```

$T = 3 + x*(4+4) = 300\ 000\ \mu s$

$x \approx 37500_{10} = 927B_H$

Subroutines



Stack Operations

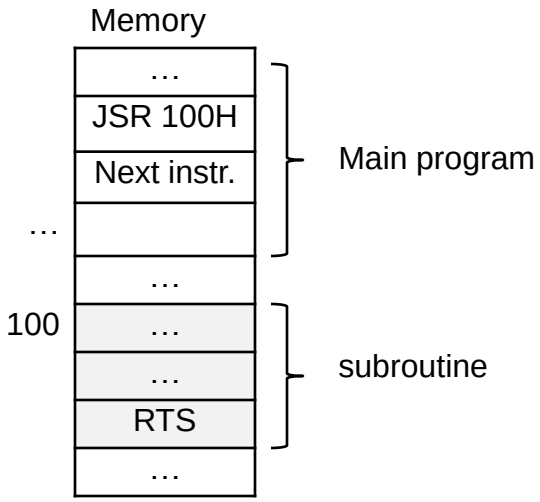
Push
PSHA
PSHB

Pop
PULA
PULB

LDS
INS
DES

Subroutines

Subroutine Call/Return

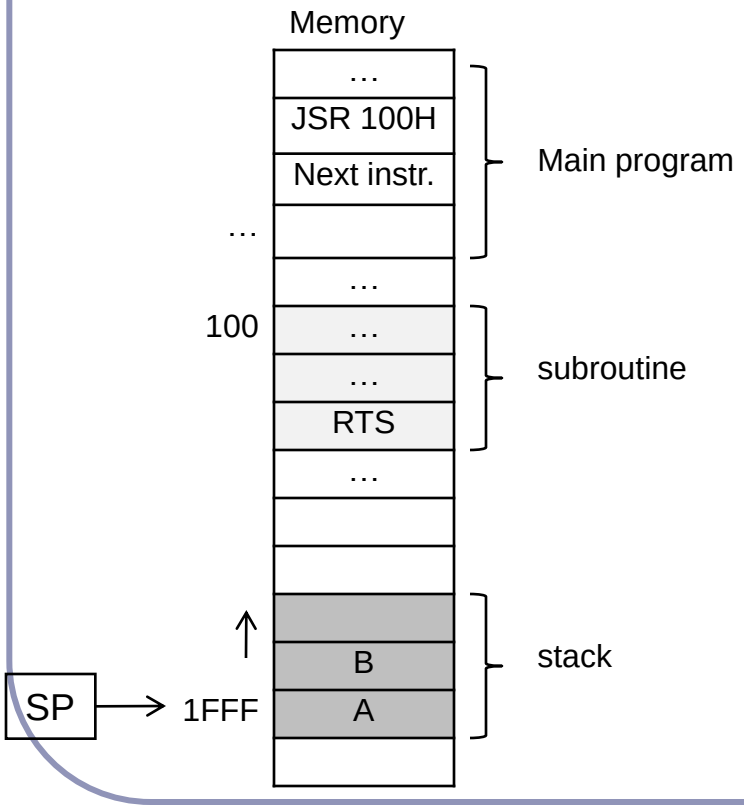


Call
JSR

Return
RTS

Subroutines

Subroutines and Stack



Ex: Write a subroutine that generates delay using two 8-bit registers. Write a main program to implement a counter that counts 0,1,2,…FF, using accumulator A, and waits 0.4 seconds before each increment.

```
        LDS   #1FFFH
        LDAA  #0H
COUNT   JSR   DELAY
        INCA
        BNE   COUNT
        END

DELAY   PSHA
        PSHB
        LDAB    #00
L1      LDAA    #00
L2      DECA
        BNE     L2
        DECB
        BNE     L1
        PULB
        PULA
        RTS
```